Provide a worst-case asymptotic time complexity of the following algorithms by using a suitable asymptotic notation considering a nearest function. Assume that there are no errors/ bugs in the algorithms. Show the meaningful working behind your answer in the rough work column.

| | | Time Complexity | Rough Work |
|---|---|---|---|
| a) | Func(n)<br>BEGIN<br>FOR (i = 1 to n)<br>FOR (j = 1 to i×i)<br>IF (j modulus i = 1) THEN<br>PRINT i<br>END FOR<br>END FOR<br>END | $O(n^3)$ | Outer loop runs n times<br>Inner loop runs i×i times<br>$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$<br>$= 1/3n^3$ |

CamScanner

$n^2 \log_3 n$

**b)**

```
for (int i = 1; i <= n*n; i = i * 3) {
    for (int j = 0; j < i; j++)
        cout << j;
}
```

$O(n^2)$

| i | j | steps |
|---|---|---|
| 1 | 1 | 1 |
| 3 | 3 | 2 |
| 9 | 9 | 4 |
| ⋮ | ⋮ | ⋮ |
| $n^2$ | $n^2$ | k → $\log_3 n$ |

$3^0 + 3^1 + 3^2 + \cdots 3^{k-1} = \frac{3^{2\log_3(n)+1} - 1}{3-1}$

$\frac{a^{n+1}-1}{a-1}$

$O(i)$

**c)**

```
for (int i = 0; i <= n; i = i+2) {
    for (int j = n; j >= i; j--) {
        for (k = 100; k >= 1; k = k/2)
            cout << k;
    }
}
```

$O(n^2)$

| i | j | step |
|---|---|---|
| 0 | n | 1 |
| 2 | n-2 | 2 |
| 4 | n-4 | 3 |
| 6 | n-6 | 4 |
| 8 | n-8 | 5 |
| ⋮ | ⋮ | ⋮ |
| n | n-n | k |

$\Sigma = 2(1+2+\cdots\frac{n}{2}) = 2\left(\frac{(n/2)(n/2+1)}{2}\right)$ → $n^2$

$(3^{\log_3(n)})^2 = n^2$

$O(1)$

**d)**

```
for (int i = 1; i*i <= n; i++)
{
    for (int j = n; j >= i*i; j--) {
        for (k = j; k >= 1; k = k/2)
            cout << k;
        for (l = 1; l <= i; l++)
            cout << l;
    }
}
```

$O(n^2)$

$i <= \sqrt{n}$ → $O(\sqrt{n})$

$j >= i^2$, $\sqrt{j} >= i$ same as above → $O(n)$

for (k=j; k>=1; k=k/2) → $\log_2(k)$

since k = j, j = n → $\log_2(n)$

| loop | time complexity |
|---|---|
| 1 | $\sqrt{n}$ |
| 2 | n |
| 3 | $\log_2(n)$ |
| 4 | $\sqrt{n}$ |

$2\sqrt{n} \cdot n(\log_2(n) + \sqrt{n}) = (n^2) + \sqrt{n}\, n \log_2(n)$

l → i, l → $\sqrt{n}$ worst case → $(\sqrt{n})$

**e)**

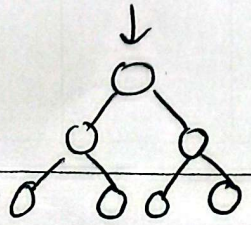
```
// Algorithm to Search in a Binary
// Search Tree
while x ≠ NIL and k ≠ x.key
    if k < x.key
        x = x.left
    else x = x.right
return x
```

$O(\log n)$

it halves everytime it compares since its a binary search tree.

$i = i/2$ everytime

so



O(Log n ) is the complexity for a balanced binary tree.
If we consider an unbalanced binary tree, the complexity will be O(n).

CamScanner

| | | | |
|---|---|---|---|
| f) | *count ← 0*<br>*for i ← 1 to ⌊√n⌋ do*<br>*if n mod i = 0 then*<br>*count ← count + 1*<br>*if i ≠ n/i then*<br>*count ← count + 1*<br>*return count* | $O(\sqrt{n})$ | i: 1, 2, 3, …, $\sqrt{n}$<br>step: 1, 2, 3, 4, 5, …, $\sqrt{n}$ |

**Question 2 [4x2=8 Marks]**

a. Use Big-O to prove or disprove that $5n^2 + 20n \log n + 100$ is $O(n^3)$.

# Question 2:

a) ~~If g(n) is O(f(n)~~
$f(n)$ is $O(g(n))$ if
$f(n) \leq c \cdot g(n)$
$f(n) = 5n^2 + 20n\log n + 100$ $g(n) = O(n^3)$
$5n^2 + 20n\log n + 100 \leq cn^3$
Dividing both side with $n^3$
$\frac{5}{n} + \frac{20}{n^2}\log n + 100 \leq cA$

for large value of n the L.H.S will be very smaller then c.
Hence $5n^2 + 20n\log n + 100$ has $O(n^3)$.

b) $2n^4 + 50n^2\log n + 7n \leq cn^4$
For $n \geq 2$
$2n^4 \leq 2n^4$
$50n^2\log n \leq 50n^2 \cdot n \leq 50n^4$
$7n \leq 7n^4$
$2n^4 + 50n^2\log n + 7n \leq 2n^4 + 50n^4 + 7n^4 = 59n^4$
Thus we have $f(n) \leq cn^4$ where $c = 59$
$f(n) = O(n^4)$

c) $n\log n + 5n + 10 \leq cn^2$
$n\log n$ For $n \geq 2$
$n\log n \leq n^2$
$5n \leq n^2$
$n\log n + 5n + 10 \leq 7n^2 + 5n^2 + 10n^2 = 16n^2$

At $c = 16$ $n_0 = 2$ $f(n) = O(n^2)$

$n \log n + 5n + 10 \geq c' n^2$

Divide b.s by $n^2$

$$\frac{\log n}{n} + \frac{5}{n} + \frac{10}{n^2} \geq c'$$

The L.H.S goes to 0 when $n \to \infty$. For large $n$ then L.H.S is smaller then any positive $c'$.

So $f(n) \neq \Omega(n^2)$

d) $2n$ is $O(n!)$

$2n \leq c \cdot n!$

$2n \leq c \cdot (n)(n-1)!$

$2 \leq c(n-1)!$

For $n \geq 2$ ~~$n \geq 3$~~ and $c = 2$ this holds

so $2n$ is $O(n!)$

# Question3:

~~1+1+1+1+1+1~~

Insert all elements of B into hash set set. The complexity of this

```
FindPair(A[n], B[m], k):
    hastable h;
    for j ← 1 to m do:
        Insert (B[j], j) into h
    for i ← 1 ton do:
        X ← A[i] - k
        if h contains X then:
            ~~j ← return(i, j)~~
            j ← index of X
            return (i, j)
    return "No such pair exist".
```

This has the time complexity of O(m+ space complexity O(m).

# Question #4

Step 1: Outer loop ($i = 0 \cdots n-1$)

- Executes $n$ times.
- Three possible branches
  - Case 1 ($arr[i] \% 3 == 0$):

$\sum_{j=0}^{n-1} j = O(n^2)$ per iteration → across $n$ iterations $O(n^3)$

  - Case 2 ($arr[i] \% 3 == 1$):

Halves each time → $O(\log n)$ per iteration → across $n$: $O(n \log n)$

  - Case 3 ($arr[i] \% 3 == 2$):

Doubling loop → $1 + 2 + 4 + \cdots + n = O(n)$ per iteration → across $n$ : $O(n^2)$

Step 2: Final Triple Nested Loop

$$\sum_{a=0}^{n-1} \sum_{b=a}^{n-1} \sum_{c=b}^{n-1} 1 = \binom{n+2}{3} = O(n^3)$$

Step 3: Combine Results

- Best Case (all elements hit Case 2): $O(n \log n) + O(n^3) = O(n^3)$
- Worst Case (all elements hit Case 1): $O(n^3) + O(n^3) = O(n^3)$
- Average Case (mixed distribution): still dominated by Case 1 + final loop $= O(n^3)$

* Dominant term: $O(n^3)$ from final triple nested loop.

# Question # 5

a) • The algorithm repeatedly scans the array forward and backwards, swapping adjacent elements only if they are out of order and their difference is atleast the current threshold.

• The threshold starts at n and decreases by 1 whenever a swap happens, but never below 1.

• In effect: it fixes large inversions first and gradually smaller ones as the threshold drops.

b) • If threshold reaches 1, all inversions are removed and array becomes fully sorted in non-decreasing order.

• If the process stops earlier, (threshold > 1), the array is only partially sorted where no adjacent pair violates both order and threshold condition.

c) Best case - O(n)

If array is already sorted or no pair with diff ≥ n only one forward + backward pass is done.

Dry run: (n = 5, A = [1, 2, 3, 4, 5])

Forward: all in order, no swap.
Backward: same.
changed = false, so exit

**Best Case: O(n)**

Worst Case - $O(n^2)$

- Many swaps occur, threshold drops to 1 → behaves like cocktail shaker (bubble sort both ways)

Dry Run ($n=5$: $A = [9, 1, 8, 2, 7]$):

- First pass, multiple swaps, threshold ↓ each time until = 1.
- Then array sorted by repeated passes.
- Each pass = $O(n)$ upto $O(n)$ passes.

**Worst Case: $O(n^2)$**

Average Case - $\Theta(n^2)$

- Random arrays usually cause threshold to drop to small values.
- Once low, algorithm sorts with cocktail shaker.

Dry Run: ($n = 4$ $A = [9, 2, 4, 1]$, threshold 4)

Forward Pass. =3.

(9,2): $9 > 2$, diff $= 7 \geq 4$ → swap → $A = [2, 9, 4, 1]$ threshold

(9,4): $9 > 4$, diff $= 5 \geq 3$ → swap → $A = [2, 4, 9, 1]$, threshold = 2

(9,1): $9 > 1$, diff $= 8 \geq 2$ → swap → $A = [2, 4, 1, 9]$, threshold = 1

Backward Pass

(1,9): in order

(4,1): $4 > 1$ diff $= 3 \geq 1$ swap $A[2, 1, 4, 9]$

(2,1): $2 > 1$. diff $= 1 \geq 1$ swap → $A[1, 2, 4, 9]$

sorted array $A = [1, 2, 4, 9]$

d) Comparison with standard sorting

- Similar to Cocktail Shaker sort (a bidirectional bubble sort) : repeatedly sweeps array in both directions.

Key difference : uses a threshold so only large differences are swapped first ; bubble/ cocktail swap all inversions.

Complexity : like bubble / cocktail → $O(n^2)$ worst / average.

## Question # 6

a) $n^2 + 50n \log n$ vs $10n^2$

Dominant term of first $= n^2$
second dominant term $= n^2$
they are of same order.
Answer: same growth.

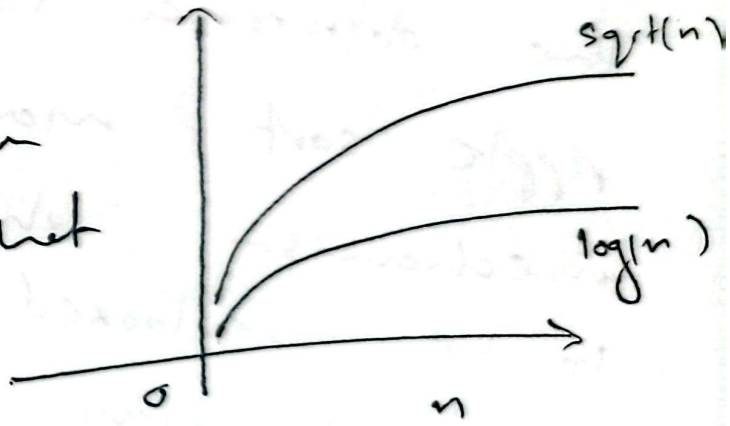
b) $n^2 \log n$ and $n^3$

$$\frac{n^2 \log(n)}{n^3} = \frac{\log(n)}{n} \qquad n > \log n.$$

since $n^3$ is the dominating term the fn $n^3$ has a greater order of growth than $n^2 \log n$.

c) $\log n$ and $\sqrt{n}$

From the graphical representation of these functions we can tell that $\sqrt{n}$ scales more than $\log n$ as $n$ increases.




$\therefore \sqrt{n}$ has higher order of growth.

c) $\log n$ & $\sqrt{n}$

$\log n$ is $O(\log n)$

$\sqrt{n}$ is $O\sqrt{n}$

F $\log n$ has lower order of growth then $\sqrt{n}$

d) $n^2$ and $2^{\log n}$

$2^{\log_2 n} = n$

So $2^{\log_2 n}$ has $O(n)$

$n^2$ is $O(n^2)$

$n^2$ has higher order of growth then $2^{\log n}$

CamScanner

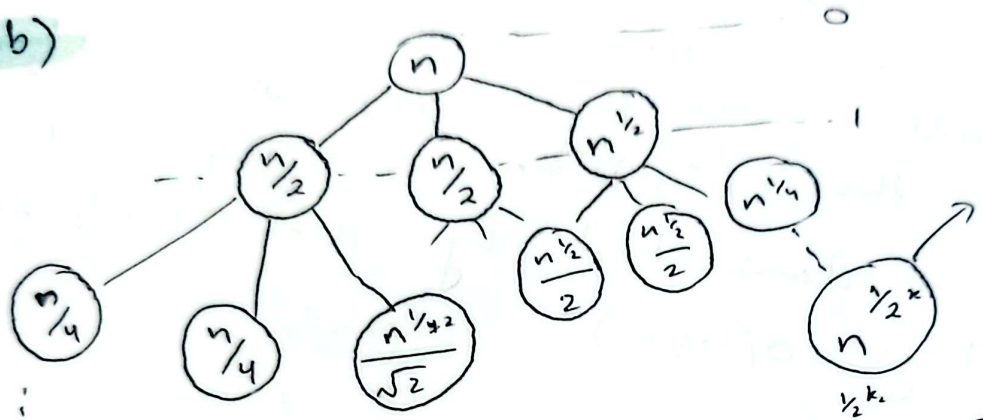
a higher order of growth.

---

## Question: 7

a) $$T(n) = \begin{cases} 2T(n/2) + T(\sqrt{n}) + n & n > 1 \\ 1 & n \le 1 \end{cases}$$

b)




these branches will be smaller since they terminate at smaller length ($\log(\log n)$)

$n^{1/2^{k_2}} \le 2$

$\frac{1}{2^{k_2}} \log n \le 1$

$k_2 = \log(\log n)$

$n/2^k$

$2^k = n$

$k = \log n$

$n/n \to 1$

longer branch since it terminates at length $k = \log n$.

Steps: $\left(\frac{n}{2} + \frac{n}{2} + \sqrt{n}\right) + 2\left(n + \sqrt{n} + \sqrt{\frac{n}{2}}\right) + \left(\frac{\sqrt{n}}{2} + \frac{\sqrt{n}}{2} + n^{1/4}\right)$

The branch $n/2$ is much longer than $\sqrt{n}$ branch since $\log n > \log(\log n)$. we can ignore $\sqrt{n}$

for worst case complexities. As n gets larger.

Complexity = $\underbrace{n+n+n\cdots}_{\log(n) \text{ times}}$

$= O(n \log(n))$.

The $T(\sqrt{n})$ term does not influence the bound since the $T(n/2)$ term branches out s larger.

---

There wouldnt be any change since any change

Question: 8

① $fun1 = \begin{cases} 1 & n<=1 \\ 2\,fun1(n/2) + n & n>1 \end{cases}$

$fun1 = 2\,fun1(n/2) + n$

$= \left[2\,fun1(n/4) + \frac{n}{2}\right] + \left[2\,fun1(n/4) + \frac{n}{2}\right] + n$

$= 4\left[fun1(n/4)\right] + \frac{n}{2} + \frac{n}{2} + n$

$= 4\left[2\,fun1(n/8) + \frac{n}{4}\right]$

$= 8\left[fun1(n/8)\right] + \underbrace{\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4}}_{n} + \underbrace{\frac{n}{2} + \frac{n}{2}}_{n} + n$

$\vdots$

$= \frac{n}{2^k}\left[fun1(n/2^k)\right] + kn$

$\therefore 2^k = n$

$k = \log n$

$= n[1] + n\log n$

∴ Complexity: $O(n \log n)$

① $$fun1 = \begin{cases} 1 & n<=1 \\ 2\,fun1(n/2) + n & n>1 \end{cases}$$

$$fun1 = 2\,fun1(n/2) + n$$

$$= \left[2\,fun1(n/4) + \frac{n}{2}\right] + \left[2\,fun1(n/4) + \frac{n}{2}\right] + n$$

$$= 4\left[fun1(n/4)\right] + \frac{n}{2} + \frac{n}{2} + n$$

$$= 4\left[2\,fun1(n/8) + \frac{n}{4}\right]$$

$$= 8\left[fun1(n/8)\right] + \underbrace{\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4}}_{n} + \underbrace{\frac{n}{2} + \frac{n}{2}}_{n} + n$$

$$\vdots$$

$$= \frac{n}{2^k} 2^k \left[fun1(n/2^k)\right] + kn$$

$\therefore 2^k = n$

$k = \log n$

$$= n[1] + n\log n$$

Complexity : $O(n \log n)$

---

② $$fun2 = \begin{cases} 1 & n=0 \\ n\,fun2(n-1) + n & n>0 \end{cases}$$

$$fun2 = n\,fun2(n-1) + n$$

$$= (n-1)\,fun2(n-2) + n(n-1) + n$$

$$= (n-1)(n-2)\,fun2(n-3) + n(n-1)(n-2) + n(n-1) + n$$

$$\vdots$$

$$= (n-1)(n-2)(n-k(n-1))\,fun2(n-n) + n(n-1)(n-2)\cdots(n-(n-1))$$

$+ \; n(n-1)(n-2)(n-3)\cdots(n-(n-2)) + \cdots + n(n-1)$

Sorted: $1 + n + n(n-1) + n(n-1)(n-2) + \cdots + n(n-1)!$

$$\frac{n!}{n!} + \frac{n!}{(n-1)!} + \frac{n!}{(n-2)!} + \cdots + \frac{n!}{3!} + \frac{n!}{2!} + \frac{n!}{1}$$

$$= n!\left(1 + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \cdots + \frac{1}{n!}\right)$$

$$= n!\left(\sum_{i=0}^{n} \frac{1}{i!} - 1\right)$$

↳ as $n \to \infty$ this value approaches $e$.

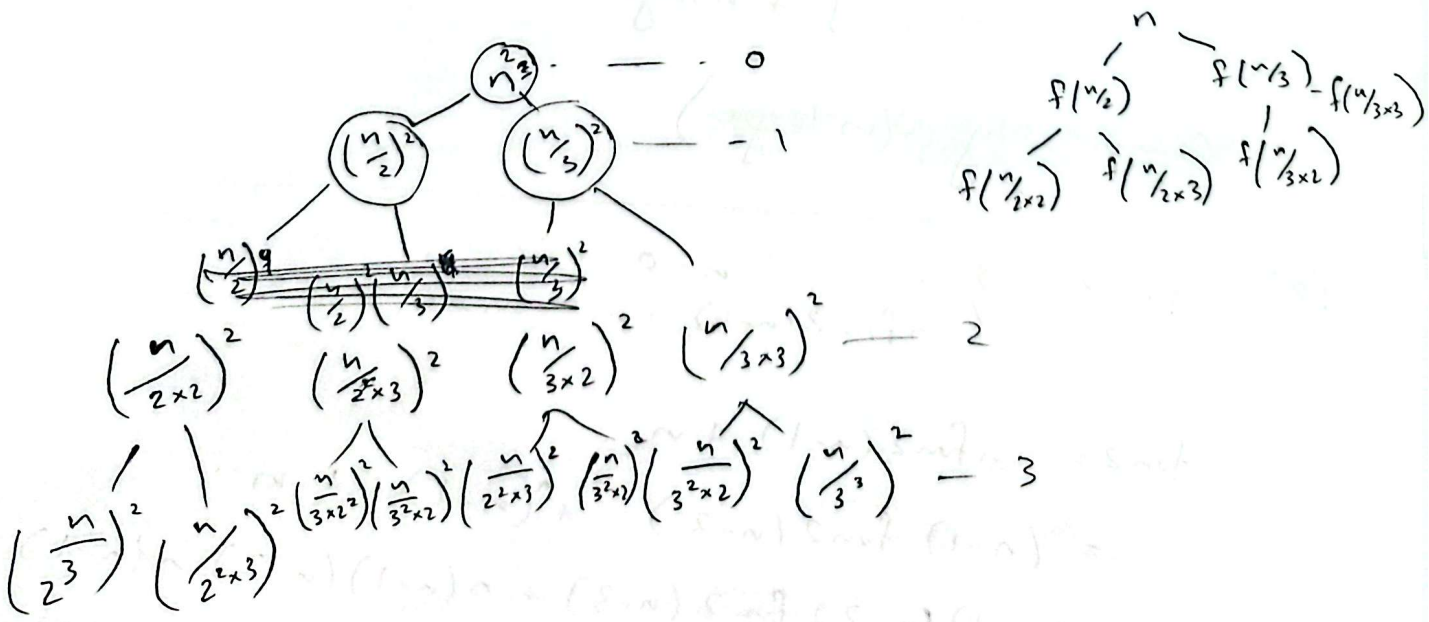
Upper bound of $\sum_{i=0}^{n} \frac{1}{i!} = e$.

$$= n!\,(e-1)$$

Complexity = $O(n!)$

---

(3) $$f = \begin{cases} 1 & n <= 2 \\ f(n/2) + f(n/3) + n^2 & n > 2 \end{cases}$$



$$n^2 + \left(\frac{n}{2}\right)^2 + \left(\frac{n}{3}\right)^2 + n^2\left(\frac{1}{2^4} + \frac{1}{2^2\times 3^2} + \frac{1}{3^2\times 2^2} + \frac{1}{3^4}\right)$$

$$= n^2 + n^2\left(\frac{1}{4} + \frac{1}{9}\right) + n^2\left(\frac{1}{4^2} + \frac{1}{9^2} + \frac{1}{4\times 9}\right)$$

$$= n^2 + n^2\left(\underbrace{\frac{1}{4} + \frac{1}{9} + \frac{1}{4^2} + \frac{1}{9^2} + \frac{2}{4\times 9}}_{\text{constant sum.}} + \cdots\right)$$

$$= cn^2$$

Complexity = $O(n^2)$

---

(4)

$$f_4 = \begin{cases} 0 & n <= 1 \\ 1 & \\ 2 f_4(n-1) + n & n > 1 \end{cases}$$

$$f(n) = 2f(n-1) + n$$

$$= 2[2F(n-2) + n-1] + n$$

$$= 4F(n-2) + 2(n-1) + n$$

$$= 4[2F(n-3) + n-2] + 2(n-1) + n$$

$$= 8F(n-3) + 4(n-2) + 2(n-1) + n$$

$$\vdots$$

$$= 2^k F(n-k) + 2^{k-1}(n-k-1) + 2^{k-2}(n-k-3) + \cdots + n$$

$$\therefore k = n$$

$$= 2^n + 2^{n-1}(n-n+1) + 2^{n-2}(n-n+2) + \cdots 2^{n-n}(n-n+n)$$

$$= 2^n + 2^{n-1} + 2^{n-2}\cdot 2 + \cdots + 2^{n-n}\, n$$

$$= 2^n\left(\frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \cdots \frac{n}{2^n}\right)$$

$$= 2^n \sum_{i=1}^{n} \frac{i}{2^i}$$ → constant sum.

Complexity = $O(2^n)$

# Question: 9

a) $T(n) = T(n-1) + n^2 \qquad T_1 = 2.$

$= T(n-2) + (n-1)^2 + n^2$

$= T(n-3) + (n-2)^2 + (n-1)^2 + n^2$

$\vdots$

$= T(n-k) + (n-(k-1))^2 + \cdots + n^2$

if $k = n-1$

$= T(n-n+1) + (n-n+2)^2 + \cdots n^2$

$= 1 + 2^2 + 3^2 + 4^2 + \cdots n^2$

$= \sum_{i=1}^{n} i^2$

$\approx \frac{1}{3} n^3 \qquad O(n^3)$

---

b) $T(n) = 2T(n/2) + 3 \quad ; \quad T_1 = 1$

$= 2\left[2T\left(\frac{n}{4}\right) + 3\right] + 3$

$= 4\left[2T\left(\frac{n}{8}\right) + 3\right] + 2 \cdot 3 + 3$

$= 8\left[2T\left(\frac{n}{16}\right) + 3\right] + 4 \cdot 3 + 2 \cdot 3 + 3.$

$\vdots$

$= 2^{k-1}\left[2T\left(\frac{n}{2^{k+1}}\right) + 3\right] + 2^{k-2} \cdot 3 + 2^{k-3} \cdot 3 + \cdots + 2^{k-k} \cdot 3$

$k = \log n$

$= 2^{\log n - 1}\left[2 + 3\right] + 2^{\log n - 2} \cdot 3 + 2^{\log n - 3} 3 \cdots + 3.$

$= 2^{\log n} + 3 \cdot (1 + 2 + 2^2 + \cdots 2^{\log n - 1})$

$= n + 3(2^{\log n} - 1)$

$= n + 3n - 3$

Complexity = $O(n)$.

---

(c) $T(n) = T(n-1) + \frac{1}{n}$ $\quad T_1 = 2$.

$T(n-1) = T(n-2) + \frac{1}{n-1}$

$T(n-2) = T(n-3) + \frac{1}{n-2}$

$T(n) = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$

$= \sum_{i=1}^{n} \frac{1}{i} = \ln(n) + \gamma$

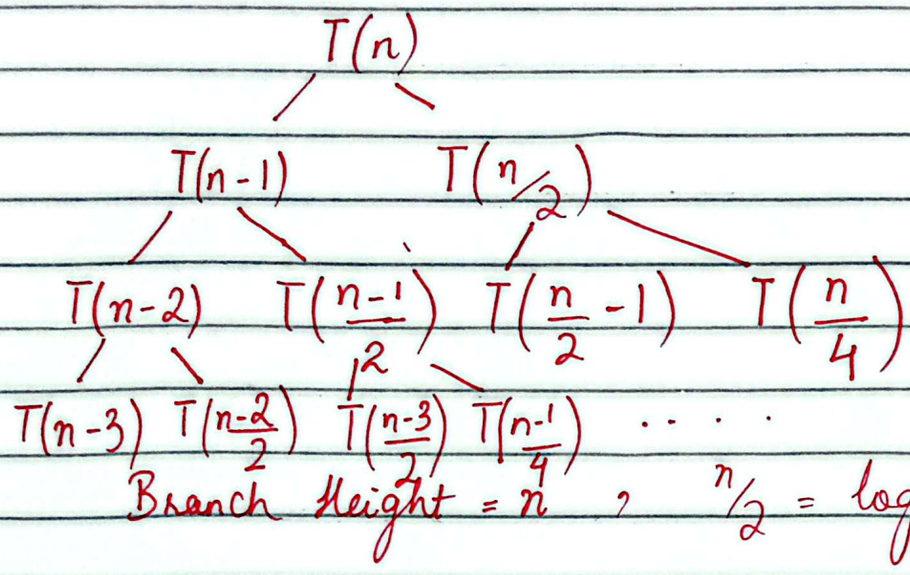
Complexity = $O(\log n)$.

CamScanner

This is one possible way to solve it, there are other ways too.

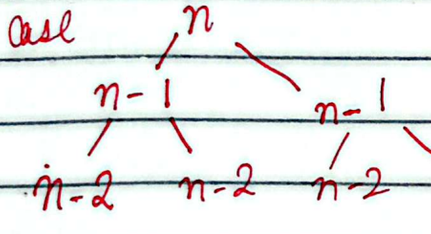
# Question # 10

a) $T(n) = T(n-1) + T\left(\frac{n}{2}\right) + n \quad , \quad T(1) = 1$




Branch Height = $n \quad , \quad n/2 = \log n$

do the worst case




$\longrightarrow n + 2(n-1) + 4(n-2) + 8(n-3) + \cdots + 2^{n-1}$

$= 2^0 n + 2^1(n-1) + 2^2(n-2) + \cdots + 2^{n-1}(1)$

$= 2^n\left(\frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \cdots \frac{n}{2^n}\right)$

$= 2^n = O(2^n)$

Iterative Method.

$$T(n) = T(n-1) + T\left(\frac{n}{2}\right) + n$$

$$T(n-1) = T(n-2) + T\left(\frac{n-1}{2}\right) + O(n-1)$$

$$T(n-2) = T(n-3) + T\left(\frac{n-2}{2}\right) + O(n-2)$$

$$T(n) = 2^2 T(n-2) + 2(n-1) + n$$

$$T(n) = 2^3 T(n-3) + 2^2(n-2) + 2(n-1) + n$$

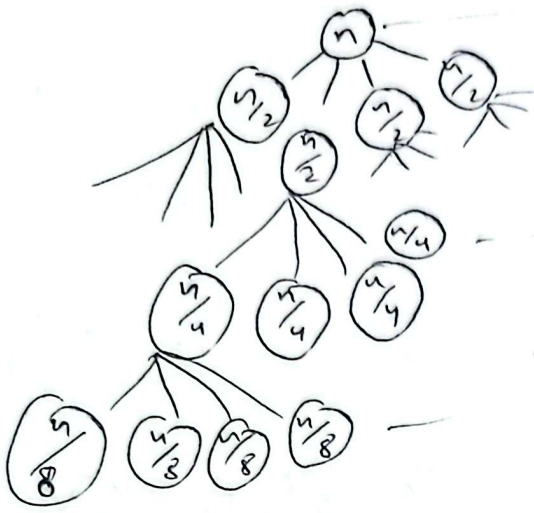
$$T(n) = 2^{n-1} + 2 \cdot 2^{n-2} + \cdots + 2(n-1) + n$$

$$\Rightarrow 2^n\left(\frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \cdots \frac{n}{2^n}\right)$$

$O(2^n)$

Q10

a) $T(n) = 4\,T(n/2) + n\log n$ $\quad T(1) = 1$




0 — $4^0$ nodes

1 — $4^1$ nodes

2 — $4^2$ nodes

3 — $4^3$ nodes

⋮

$4^{\log n}$ nodes.

$\frac{n}{2^k}$ $\quad k = \log n$.

$$\text{sum} = n\log n + 4\left(\frac{n}{2}\log\frac{n}{2}\right) + 4^2\left(\frac{n}{2^2}\log\frac{n}{2^2}\right) + \cdots + 4^3\left(\frac{n}{2^3}\log\frac{n}{2^3}\right)$$

$$= n\log n + 2n\log\frac{n}{2} + 2^2 n\log\frac{n}{2^2} + \cdots + 2^k n\log\left(\frac{n}{2^k}\right)$$

$$= \sum_{i=0}^{\log n - 1} 2^i n\log\left(\frac{n}{2^i}\right)$$

$$n\left(\sum 2^i \log n - \sum 2^i\right) \qquad 2^{\log n} = n$$

~~$n\,(n\log n - n)$~~

$$= n\left\{\log n\,(2^k - 1) - \left((k-2)\,2^k + 2\right)\right\}$$

$$= n\left\{\log n\,(n-1) - (\log n \cdot 2n + 2)\right\}$$

$$= 2n^2 - n\log n + 2n$$

Complexity = $O(n^2)$.

Iterative:

$$T(n) = 4T\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^2 T\left(\frac{n}{2^2}\right) + 2n\log\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^3 T\left(\frac{n}{2^3}\right) + 2^2 n \log\left(\frac{n}{2^2}\right) + \cdots n\log n$$

$$= 4^k T\left(\frac{n}{2^k}\right) + 2^{k-1} n \log\left(\frac{n}{2^{k-1}}\right) + \cdots n\log n$$

$$= n^2 T(1) + \sum_{i=0}^{\log n - 1} 2^i n \log\left(\frac{n}{2^i}\right)$$

same term as last result.

$$= n^2 + (2n^2 - \log(n) + 2n$$

$O(n^2)$ complexity.

Iterative:

$$T(n) = 4T\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^2 T\left(\frac{n}{2^2}\right) + 2n\log\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^3 T\left(\frac{n}{2^3}\right) + 2^2 n\log\left(\frac{n}{2^2}\right) + \dots$$

$$= 4^k T\left(\frac{n}{2^k}\right) + 2^{k-1} n\log\left(\frac{n}{2^{k-1}}\right) + \dots$$

$$= n^2 T(1) + \sum_{i=0}^{\log n - 1} 2^i n\log\left(\frac{n}{2^i}\right)$$

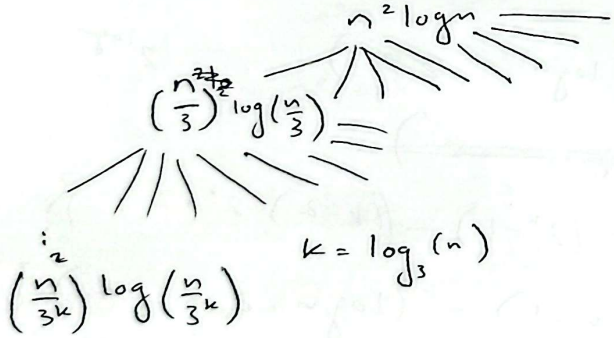
same term as last result.

$$= n^2 + (2n^2 - \log(n) + 2n$$

$O(n^2)$ complexity.

---

(c) $9T(n/3) + n^2\log n$.




Sum: $n^2 \sum_{i=0}^{k-1} \log\left(\frac{n}{3^i}\right)$

$n^2\left(\sum \log n - \sum 3^i\right)$

$n^2\left(\frac{\log^2(n) + \log(n)}{3-1}\right)$

Complexity: $O(n^2\log^2(n))$

Iterative

$$T(n) = 4T\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^2 T\left(\frac{n}{2^2}\right) + 2n\log\left(\frac{n}{2}\right) + n\log(n)$$

$$= 4^3 T\left(\frac{n}{2^3}\right) + 2^2 n\log\left(\frac{n}{2^2}\right) + \cdots$$

$$= 4^k T\left(\frac{n}{2^k}\right) + 2^{k-1} n\log\left(\frac{n}{2^{k-1}}\right) + \cdots$$

$$= n^2 T(1) + \sum_{i=0}^{\log n - 1} 2^i n\log\left(\frac{n}{2^i}\right)$$

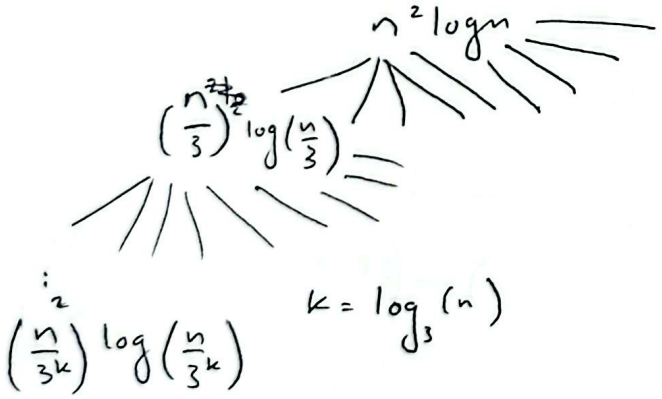
same term as last result.

$$= n^2 + (2n^2 - \log(n) + 2n$$

$O(n^2)$ complexity.

---

(c) $9T(n/3) + n^2\log n$.




Sum. $n^2 \sum_{i=0}^{k-1} \log\left(\frac{n}{3^i}\right)$

$n^2\left(\sum \log n - \sum 3^i\right)$

$n^2\left(\frac{\log^2(n) + \log(n)}{3-1}\right)$

Complexity: $\Theta(n^2\log^2(n))$

Iterative: oh my god

$$T(n) = 9\,T(n/3) + n^2 \log n$$

$$T(n/3) = 9^2\,T(n/9) + 3^2 n^2 \log\left(\frac{n}{3}\right) + n^2 \log n$$

$$T(n/9) = 9^3\,T(n/27) + 9^2 n^2 \log\left(\frac{n}{9}\right) + \cdots n^2 \log n$$

$$= 9^k\,T\left(n/3^k\right) + n^2 \log\left(\frac{n}{3^{k-1}}\right) + \cdots n^2 \log n$$

$k = \log_3 n$

$$= 9^{\log_3 n} + n^2 \left(\sum_{i=0}^{k=1} \log\left(\frac{n}{3^i}\right)\right)$$

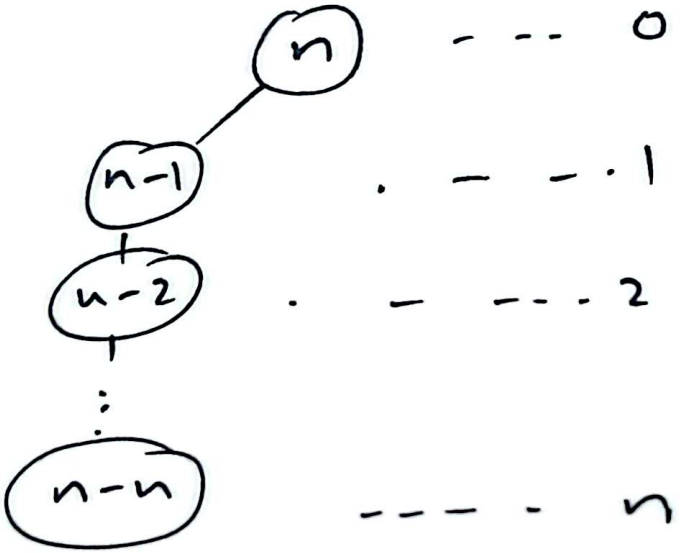
$$= n^2 + n^2 \left(\sum_{i=0}^{\log n} \log n - \sum i\right)$$

$$= n^2 + n^2 \log^2 n - n^2\left(\frac{\log^2 n + \log n}{2}\right)$$

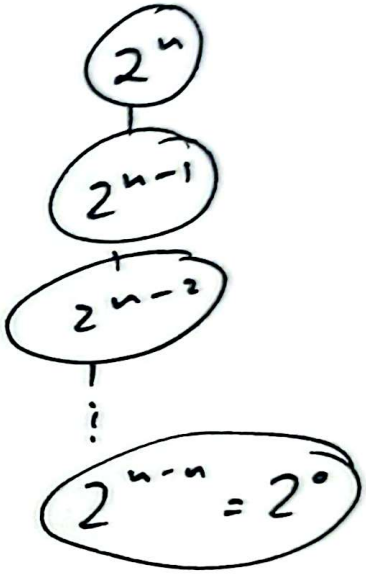
$$= n^2 + n^2 \log^2 n - \frac{n^2 \log^2 n}{2} + \frac{n^2 \log n}{2}$$

Complexity = $O(n^2 \log^2 n)$.

d) $T(n) = T(n-1) + 2^n$ $\quad T(0) = 1$




fun. call graph




step graph.

$$\text{Sum} = 2^0 + 2^1 + 2^2 + \cdots 2^n$$

$$= 2^n \underbrace{\left( \frac{1}{1} + \frac{1}{2} + \frac{1}{2^2} + \cdots \frac{1}{2^n} \right)}_{\text{constant}}$$

Complexity = $\Theta(2^n)$

CamScanner

Iterative : $T(n) = T(n-1) + 2^n$

$= T(n-2) + 2^{n-1} + 2^n$

$= T(n-3) + 2^{n-2} + 2^{n-1} + 2^n$

$\vdots$

$= T(n-n) + 2^1 + 2^2 + \cdots + 2^n$

$= 2^n \left( \underbrace{\frac{1}{4} + \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots \frac{1}{2^n}}_{\text{constant.}} \right)$

$= 2^n$

Complexity = $O(\approx 2^n)$.

X ——————————————— X